

BABU BANARSI DAS UNIVERSITY



SCHOOL OF COMPUTER APPLICATIONS Assignment of NO SQL Lab File

Submitted to

Harendra Singh

Submitted by

**Saumya
BCADS-26**

11

NoSQL Database Project — MongoDB

0. MongoDB Installation & Setup

Before performing the database operations, MongoDB Community Server and MongoDB Shell (mongosh) should be installed on the system. After installation, the following commands can be used to verify that MongoDB is available.

Installation Steps

1. Download and install MongoDB Community Server for Windows.
2. During installation, keep the default setup options unless a different location is required.
3. Install MongoDB Shell (mongosh) if it is not included with the selected installation.
4. Open Command Prompt or PowerShell after installation.

MongoDB Verification Commands

```
mongod --version  
mongosh --version
```

Explanation of Output: The commands display the installed MongoDB server and MongoDB Shell versions. A version number confirms that the corresponding component is installed and available from the command line.

Starting MongoDB Shell

```
mongosh
```

Explanation of Output: The mongosh command opens the MongoDB Shell, where database commands such as use, insertMany(), find(), updateOne(), deleteOne(), sorting, and counting can be executed.

1. Create / Insert Operation

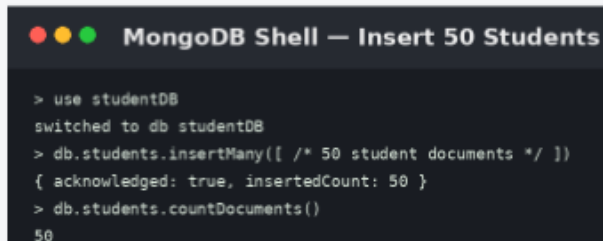
The insertMany() operation is used to create and store multiple student documents in a MongoDB collection. In this project, 50 student records are inserted into the students collection. Each document contains a unique student ID and fields such as name, age, gender, course, marks, city and fees_paid.

MongoDB Query

```
use studentDB

db.students.insertMany([
  {student_id:1, name:"Aarav Sharma", age:21, gender:"Female",
  course:"BCA", marks:62, city:"Lucknow", fees_paid:25000},
  {student_id:2, name:"Aditya Verma", age:18, gender:"Male",
  course:"BBA", marks:69, city:"Kanpur", fees_paid:30000},
  // ... continue the same structure for all 50 students ...
  {student_id:50, name:"Meera Joshi", age:20, gender:"Female",
  course:"BCA", marks:96, city:"Lucknow", fees_paid:25000}
])
```

Output Screenshot



The screenshot shows a terminal window titled "MongoDB Shell — Insert 50 Students". The commands and their outputs are as follows:

```
> use studentDB
switched to db studentDB
> db.students.insertMany([ /* 50 student documents */ ])
{ acknowledged: true, insertedCount: 50 }
> db.students.countDocuments()
50
```

Explanation of Output: The result shows acknowledged: true and insertedCount: 50, confirming that all 50 student documents were added successfully.

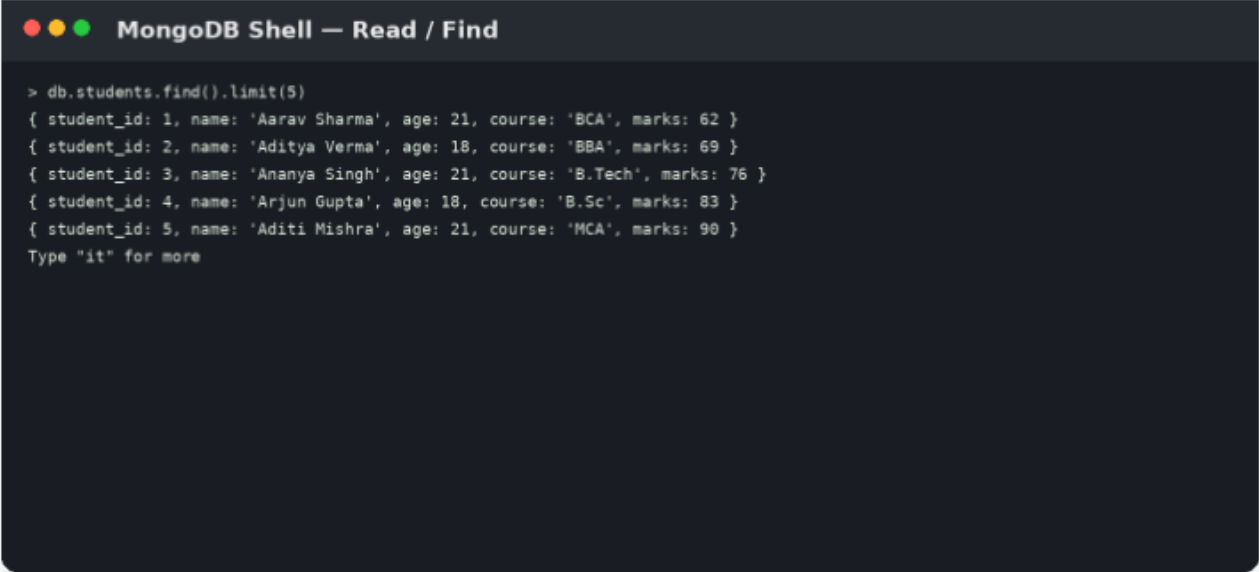
2. Read / Find Operation

The `find()` operation is used to retrieve documents from a MongoDB collection. It can return all documents or only documents that satisfy a condition. The `limit()` method restricts the number of documents displayed.

MongoDB Query

```
db.students.find().limit(5)
```

Output Screenshot



The screenshot shows a terminal window titled "MongoDB Shell — Read / Find". The command `> db.students.find().limit(5)` has been executed, resulting in the following JSON output:

```
{ student_id: 1, name: 'Aarav Sharma', age: 21, course: 'BCA', marks: 62 }  
{ student_id: 2, name: 'Aditya Verma', age: 18, course: 'BBA', marks: 69 }  
{ student_id: 3, name: 'Ananya Singh', age: 21, course: 'B.Tech', marks: 76 }  
{ student_id: 4, name: 'Arjun Gupta', age: 18, course: 'B.Sc', marks: 83 }  
{ student_id: 5, name: 'Aditi Mishra', age: 21, course: 'MCA', marks: 90 }
```

Below the output, the text "Type 'it' for more" is displayed.

Explanation of Output: The output displays the first five student documents. This demonstrates that the records can be read after insertion.

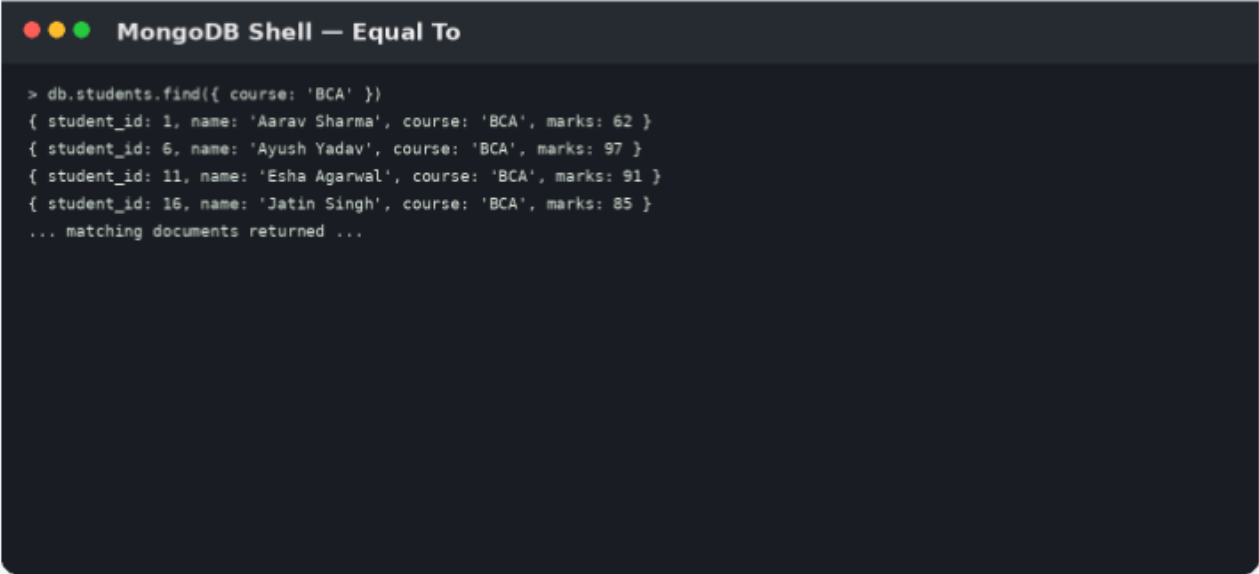
3. Equal To (=) Operation

MongoDB uses a field-value pair for an equality condition. It returns documents where the specified field exactly matches the given value. Here, the query searches for students whose course is BCA.

MongoDB Query

```
db.students.find({ course: "BCA" })
```

Output Screenshot



The screenshot shows a terminal window titled "MongoDB Shell — Equal To". It displays the execution of a MongoDB query and its output. The query is `> db.students.find({ course: 'BCA' })`. The output shows four documents as JSON objects, each with fields for `student_id`, `name`, `course`, and `marks`. The documents are for Aarav Sharma (marks: 62), Ayush Yadav (marks: 97), Esha Agarwal (marks: 91), and Jatin Singh (marks: 85). The output concludes with `... matching documents returned ...`.

```
> db.students.find({ course: 'BCA' })
{ student_id: 1, name: 'Aarav Sharma', course: 'BCA', marks: 62 }
{ student_id: 6, name: 'Ayush Yadav', course: 'BCA', marks: 97 }
{ student_id: 11, name: 'Esha Agarwal', course: 'BCA', marks: 91 }
{ student_id: 16, name: 'Jatin Singh', course: 'BCA', marks: 85 }
... matching documents returned ...
```

Explanation of Output: Only documents whose course field is exactly BCA are returned.

4. Greater Than (\$gt) Operation

The \$gt operator means greater than. It is useful when records need to be filtered above a particular numeric value. The following query finds students whose marks are greater than 90.

MongoDB Query

```
db.students.find({ marks: { $gt: 90 } })
```

Output Screenshot

MongoDB Shell — Greater Than (\$gt)

```
> db.students.find({ marks: { $gt: 90 } })
{ student_id: 6, name: 'Ayush Yadav', marks: 97 }
{ student_id: 13, name: 'Gauri Joshi', marks: 95 }
{ student_id: 20, name: 'Manav Jain', marks: 95 }
{ student_id: 27, name: 'Riya Yadav', marks: 97 }
... matching documents returned ...
```

Explanation of Output: The output contains students with marks strictly greater than 90. A student with exactly 90 marks would not be included.

5. Less Than (\$lt) Operation

The \$lt operator means less than. It filters records whose numeric value is smaller than the specified value. The example below finds students younger than 20 years.

MongoDB Query

```
db.students.find({ age: { $lt: 20 } })
```

Output Screenshot

MongoDB Shell — Less Than (\$lt)

```
> db.students.find({ age: { $lt: 20 } })
{ student_id: 2, name: 'Aditya Verma', age: 18 }
{ student_id: 4, name: 'Arjun Gupta', age: 18 }
{ student_id: 6, name: 'Ayush Yadav', age: 18 }
{ student_id: 8, name: 'Chetan Kumar', age: 18 }
... matching documents returned ...
```

Explanation of Output: The result contains students whose age is strictly below 20.

6. Greater Than or Equal To (\$gte)

The \$gte operator means greater than or equal to. Unlike \$gt, it also includes records whose value is exactly equal to the specified limit.

MongoDB Query

```
db.students.find({ marks: { $gte: 85 } })
```

Output Screenshot

MongoDB Shell — Greater Than or Equal (\$gte)

```
> db.students.find({ marks: { $gte: 85 } })
{ student_id: 5, name: 'Aditi Mishra', marks: 90 }
{ student_id: 6, name: 'Ayush Yadav', marks: 97 }
{ student_id: 9, name: 'Diya Patel', marks: 98 }
{ student_id: 10, name: 'Dhruv Saxena', marks: 55 } // excluded
... matching documents returned ...
```

Explanation of Output: Students with marks of 85 or above are selected.

7. Less Than or Equal To (\$lte)

The \$lte operator means less than or equal to. It includes records below the limit as well as records exactly equal to the limit.

MongoDB Query

```
db.students.find({ marks: { $lte: 60 } })
```

Output Screenshot

MongoDB Shell — Less Than or Equal (\$lte)

```
> db.students.find({ marks: { $lte: 60 } })
{ student_id: 10, name: 'Dhruv Saxena', marks: 55 }
{ student_id: 14, name: 'Harsh Tiwari', marks: 57 }
{ student_id: 24, name: 'Pankaj Verma', marks: 55 }
{ student_id: 28, name: 'Rohan Gupta', marks: 51 }
... matching documents returned ...
```

Explanation of Output: The query returns students whose marks are 60 or less.

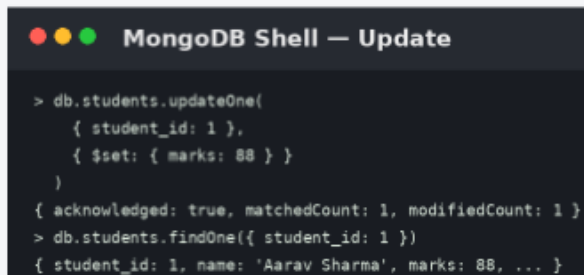
8. Update Operation

The `updateOne()` operation changes an existing document. The `$set` operator is used to assign a new value to a field without replacing the complete document.

MongoDB Query

```
db.students.updateOne(  
  { student_id: 1 },  
  { $set: { marks: 88 } }  
)
```

Output Screenshot



```
MongoDB Shell — Update  
  
> db.students.updateOne(  
  { student_id: 1 },  
  { $set: { marks: 88 } }  
)  
{ acknowledged: true, matchedCount: 1, modifiedCount: 1 }  
> db.students.findOne({ student_id: 1 })  
{ student_id: 1, name: 'Aarav Sharma', marks: 88, ... }
```

Explanation of Output: The output shows `matchedCount: 1` and `modifiedCount: 1`, confirming that the selected student's marks were updated.

9. Delete Operation

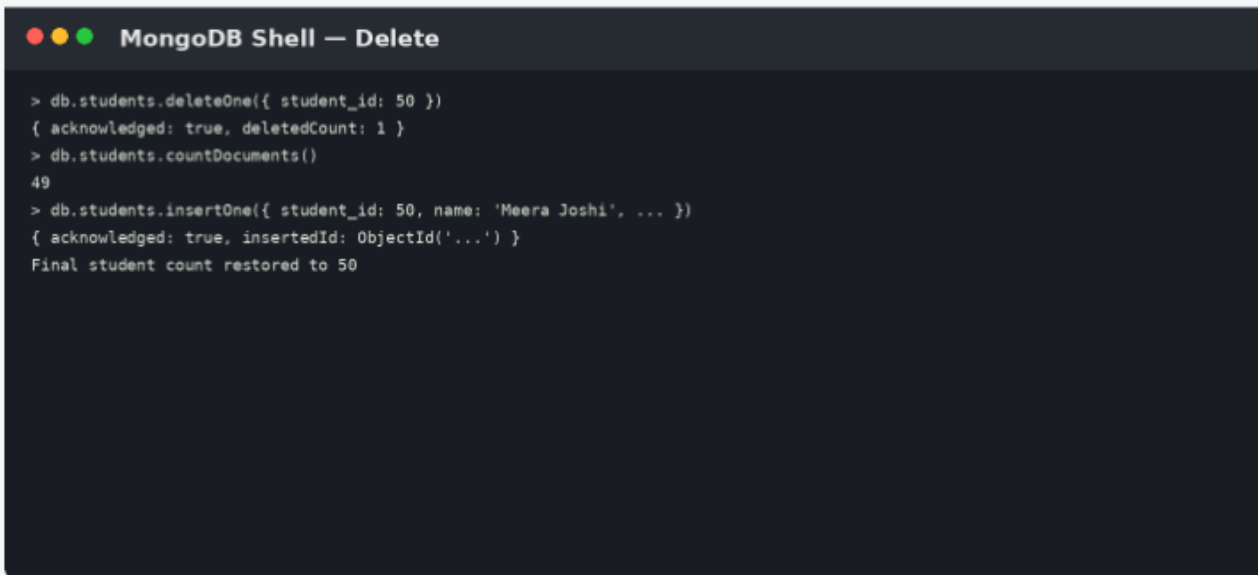
The `deleteOne()` operation removes one document that matches the specified condition. This example deletes the student with `student_id` 50. The record is then reinserted so the project again contains 50 students.

MongoDB Query

```
db.students.deleteOne({ student_id: 50 })

// Optional restoration
db.students.insertOne({
  student_id: 50, name: "Meera Joshi", age: 20,
  gender: "Female", course: "BCA", marks: 96,
  city: "Lucknow", fees_paid: 25000
})
```

Output Screenshot

A screenshot of a terminal window titled "MongoDB Shell — Delete". The terminal shows the following commands and their outputs: 1. Command: `> db.students.deleteOne({ student_id: 50 })`. Output: `{ acknowledged: true, deletedCount: 1 }`. 2. Command: `> db.students.countDocuments()`. Output: `49`. 3. Command: `> db.students.insertOne({ student_id: 50, name: 'Meera Joshi', ... })`. Output: `{ acknowledged: true, insertedId: ObjectId('...') }`. 4. A final line of text: `Final student count restored to 50`.

```
MongoDB Shell — Delete

> db.students.deleteOne({ student_id: 50 })
{ acknowledged: true, deletedCount: 1 }
> db.students.countDocuments()
49
> db.students.insertOne({ student_id: 50, name: 'Meera Joshi', ... })
{ acknowledged: true, insertedId: ObjectId('...') }
Final student count restored to 50
```

Explanation of Output: `deletedCount: 1` confirms that one document was removed. The restoration query brings the collection back to 50 records.

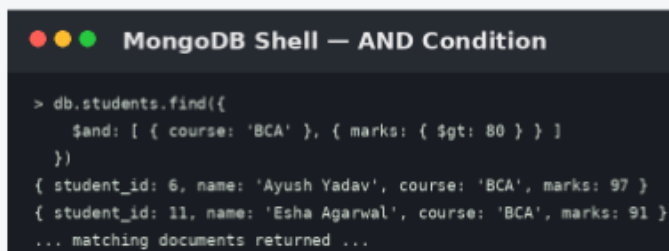
10. AND Operation (\$and)

The \$and logical operator requires all specified conditions to be true. Here, the query selects students who are in the BCA course and also have marks greater than 80.

MongoDB Query

```
db.students.find({
  $and: [
    { course: "BCA" },
    { marks: { $gt: 80 } }
  ]
})
```

Output Screenshot



The screenshot shows a terminal window titled "MongoDB Shell — AND Condition". It displays the execution of a MongoDB query using the \$and operator. The query filters for students in the 'BCA' course with marks greater than 80. The output shows two matching documents: one for 'Ayush Yadav' with 97 marks and another for 'Esha Agarwal' with 91 marks. The terminal text is as follows:

```
> db.students.find({
  $and: [ { course: 'BCA' }, { marks: { $gt: 80 } } ]
})
{ student_id: 6, name: 'Ayush Yadav', course: 'BCA', marks: 97 }
{ student_id: 11, name: 'Esha Agarwal', course: 'BCA', marks: 91 }
... matching documents returned ...
```

Explanation of Output: Only students satisfying both conditions are displayed.

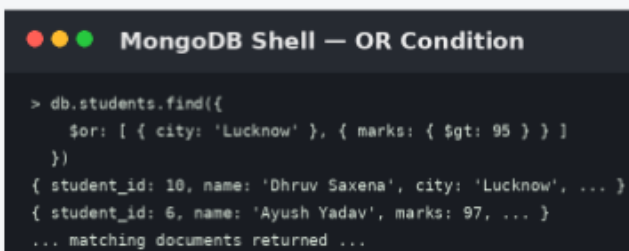
11. OR Operation (\$or)

The \$or logical operator returns a document when at least one of the supplied conditions is true. The example searches for students from Lucknow or students scoring more than 95.

MongoDB Query

```
db.students.find({
  $or: [
    { city: "Lucknow" },
    { marks: { $gt: 95 } }
  ]
})
```

Output Screenshot



The screenshot shows a terminal window titled "MongoDB Shell — OR Condition". It displays the execution of a MongoDB query using the \$or operator. The query searches for documents in the 'students' collection where either the city is 'Lucknow' or the marks are greater than 95. The output shows two documents: one for Dhruv Saxena (city: Lucknow) and one for Ayush Yadav (marks: 97). The terminal text is as follows:

```
> db.students.find({
  $or: [ { city: 'Lucknow' }, { marks: { $gt: 95 } } ]
})
{ student_id: 10, name: 'Dhruv Saxena', city: 'Lucknow', ... }
{ student_id: 6, name: 'Ayush Yadav', marks: 97, ... }
... matching documents returned ...
```

Explanation of Output: The output contains documents that satisfy either the first condition, the second condition, or both.

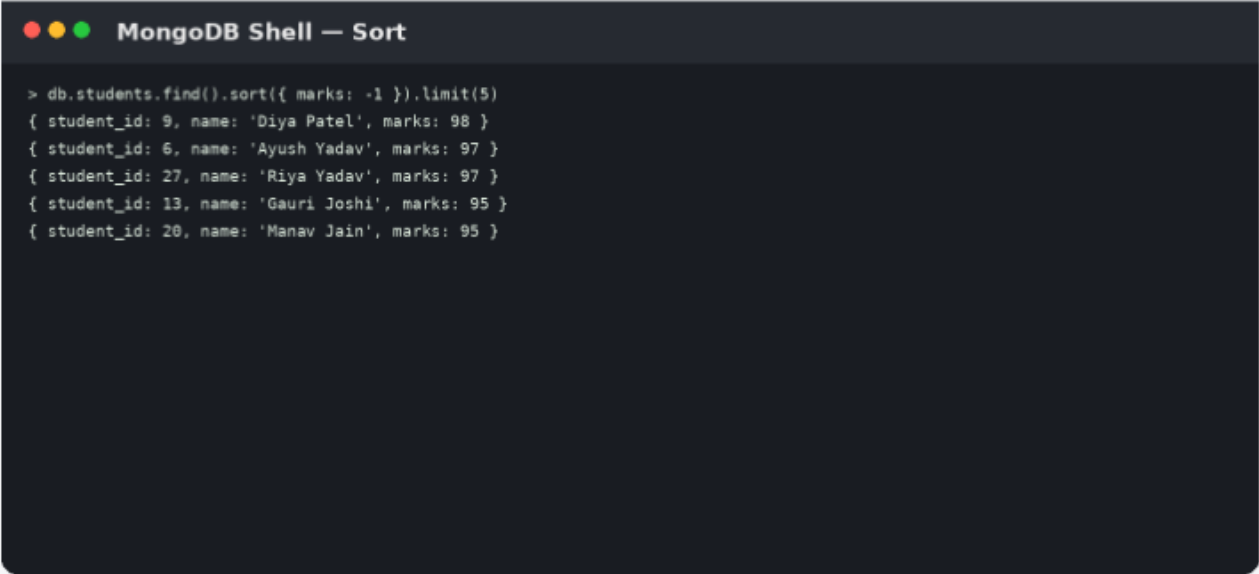
12. Sort Operation

The `sort()` method arranges documents according to one or more fields. A value of 1 sorts in ascending order and -1 sorts in descending order. Here marks are sorted from highest to lowest.

MongoDB Query

```
db.students.find().sort({ marks: -1 }).limit(5)
```

Output Screenshot



The screenshot shows a terminal window titled "MongoDB Shell — Sort". It displays the execution of the query `db.students.find().sort({ marks: -1 }).limit(5)` and its output, which lists five student records sorted by marks in descending order.

```
> db.students.find().sort({ marks: -1 }).limit(5)
{ student_id: 9, name: 'Diya Patel', marks: 98 }
{ student_id: 6, name: 'Ayush Yadav', marks: 97 }
{ student_id: 27, name: 'Riya Yadav', marks: 97 }
{ student_id: 13, name: 'Gauri Joshi', marks: 95 }
{ student_id: 20, name: 'Manav Jain', marks: 95 }
```

Explanation of Output: The five highest-mark records are displayed first because marks are sorted in descending order.

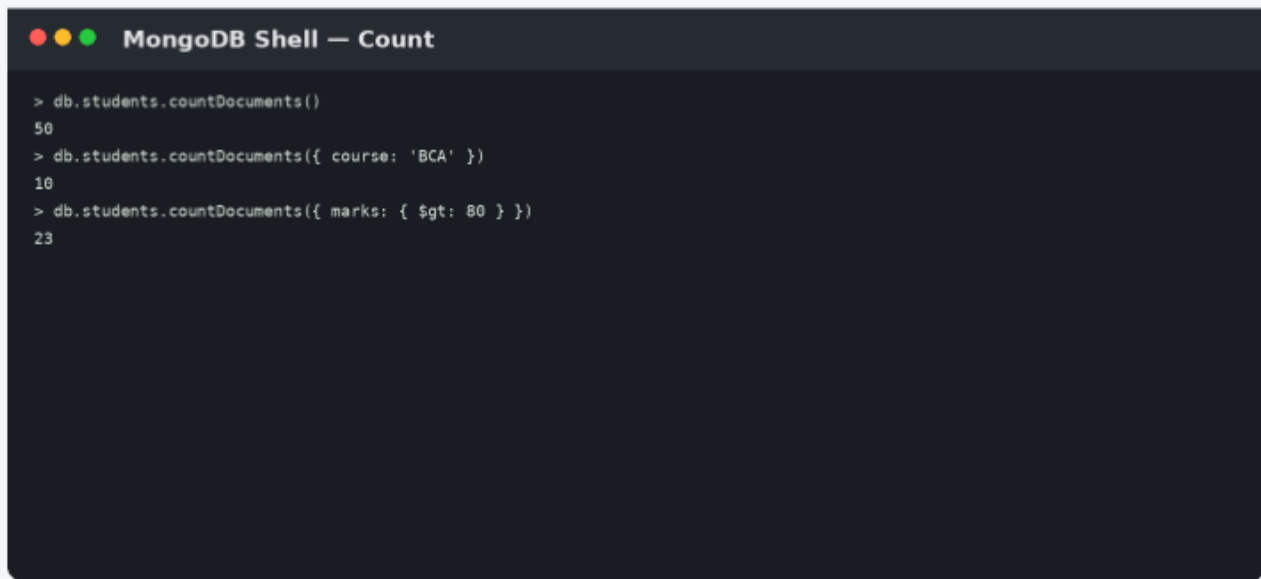
13. Count Operation

The `countDocuments()` method counts documents in a collection. It can also count only documents matching a condition. This is useful for quickly measuring the size of a dataset or a filtered group.

MongoDB Query

```
db.students.countDocuments()  
  
db.students.countDocuments({ course: "BCA" })  
  
db.students.countDocuments({ marks: { $gt: 80 } })
```

Output Screenshot

A screenshot of a terminal window titled "MongoDB Shell — Count". The terminal shows three MongoDB queries and their corresponding counts. The first query, `db.students.countDocuments()`, returns 50. The second query, `db.students.countDocuments({ course: 'BCA' })`, returns 10. The third query, `db.students.countDocuments({ marks: { $gt: 80 } })`, returns 23.

```
MongoDB Shell — Count  
  
> db.students.countDocuments()  
50  
> db.students.countDocuments({ course: 'BCA' })  
10  
> db.students.countDocuments({ marks: { $gt: 80 } })  
23
```

Explanation of Output: The output gives the total number of students, the number of BCA students, and the number of students scoring above 80.